Python Code

Project: Enterprise Customer Churn Prediction & Advanced Artificial Intelligence Analytics

Student: JATIN

Note: The project uses synthetic practice data. Results should not be presented as real-world findings.

Source code

```
"""
Enterprise Customer Churn Prediction & Advanced Artificial Intelligence Analytics
Prepared by: JATIN

IMPORTANT:
This script uses generated synthetic practice data to demonstrate a complete ML workflow.
It does not use real customer records and its metrics must not be represented as real-world results.
Replace the synthetic-data function with an approved public/mentor-provided dataset if required.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)

RANDOM_STATE = 42
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)


def make_synthetic_dataset(n=1200, random_state=RANDOM_STATE):
    """Create a reproducible synthetic dataset for demonstration only."""
    rng = np.random.default_rng(random_state)
    tenure = rng.integers(1, 73, size=n)
    monthly_charges = np.round(rng.uniform(25, 120, size=n), 2)
    contract = rng.choice(
        ["Month-to-month", "One year", "Two year"],
        size=n, p=[0.56, 0.25, 0.19]
    )
    internet_service = rng.choice(
        ["DSL", "Fiber optic", "No"], size=n, p=[0.34, 0.48, 0.18]
    )
    support = rng.choice(["Yes", "No"], size=n, p=[0.42, 0.58])

    # Simulated probability patterns are deliberately illustrative, not empirical.
    logit = (
        -1.45
        + 1.05 * (contract == "Month-to-month")
        + 0.40 * (contract == "One year")
        + 0.75 * (tenure < 12)
        + 0.35 * (monthly_charges > 90)
        + 0.28 * (internet_service == "Fiber optic")
        + 0.40 * (support == "No")
        - 0.012 * tenure
    )
    probability = 1 / (1 + np.exp(-logit))
    churn = np.where(rng.random(n) < probability, "Yes", "No")

    return pd.DataFrame({
        "CustomerID": [f"C{i:05d}" for i in range(1, n + 1)],
        "TenureMonths": tenure,
        "MonthlyCharges": monthly_charges,
        "Contract": contract,
        "InternetService": internet_service,
        "TechSupport": support,
        "Churn": churn
```

```
    })


def main():
    df = make_synthetic_dataset()
    df.to_csv(OUTPUT_DIR / "synthetic_churn_data.csv", index=False)

    print("ENTERPRISE CUSTOMER CHURN PREDICTION")
    print("Prepared by: JATIN")
    print("Dataset note: generated synthetic practice data, not real customer data.")
    print(f"Rows and columns: {df.shape}")
    print("\nMissing values by column:\n", df.isna().sum())
    print("Duplicate rows:", int(df.duplicated().sum()))

    churn_rate = (df["Churn"].eq("Yes").mean()) * 100
    print(f"\nSynthetic-data churn rate: {churn_rate:.2f}%")
    print("Average monthly charges:", round(df["MonthlyCharges"].mean(), 2))
    print("Average tenure (months):", round(df["TenureMonths"].mean(), 2))

    target = df["Churn"].map({"No": 0, "Yes": 1})
    features = df.drop(columns=["CustomerID", "Churn"])
    numeric_features = ["TenureMonths", "MonthlyCharges"]
    categorical_features = ["Contract", "InternetService", "TechSupport"]

    numeric_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    categorical_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ])
    preprocessor = ColumnTransformer([
        ("numeric", numeric_pipe, numeric_features),
        ("categorical", categorical_pipe, categorical_features)
    ])
    model = Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000, class_weight="balanced",
                                          random_state=RANDOM_STATE))
    ])

    X_train, X_test, y_train, y_test = train_test_split(
        features, target, test_size=0.25, random_state=RANDOM_STATE,
        stratify=target
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    probability = model.predict_proba(X_test)[:, 1]

    metrics = {
        "accuracy": float(accuracy_score(y_test, pred)),
        "precision": float(precision_score(y_test, pred, zero_division=0)),
        "recall": float(recall_score(y_test, pred, zero_division=0)),
        "f1_score": float(f1_score(y_test, pred, zero_division=0)),
        "roc_auc": float(roc_auc_score(y_test, probability)),
        "test_rows": int(len(y_test)),
        "dataset_type": "synthetic demonstration data"
    }
    (OUTPUT_DIR / "evaluation_metrics.json").write_text(
        json.dumps(metrics, indent=2), encoding="utf-8"
    )
    print("\nEvaluation metrics (synthetic demonstration only):")
    for key, value in metrics.items():
        if isinstance(value, float):
            print(f"{key}: {value:.3f}")
        else:
            print(f"{key}: {value}")
    print("\nClassification report:\n", classification_report(
        y_test, pred, target_names=["Retained", "Churned"], zero_division=0
    ))

    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(5, 4))
    plt.imshow(cm)
    plt.title("Confusion Matrix (Synthetic Demonstration Data)")
    plt.xlabel("Predicted label")
    plt.ylabel("Actual label")
    plt.xticks([0, 1], ["Retained", "Churned"])
    plt.yticks([0, 1], ["Retained", "Churned"])
    for (i, j), value in np.ndenumerate(cm):
```

```
            plt.text(j, i, str(value), ha="center", va="center")
        plt.tight_layout()
        plt.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=160)
        plt.close()

        churn_by_contract = pd.crosstab(df["Contract"], df["Churn"], normalize="index") * 100
        churn_by_contract.to_csv(OUTPUT_DIR / "churn_rate_by_contract.csv")
        churn_by_contract.reindex(columns=["No", "Yes"], fill_value=0).plot(
            kind="bar", figsize=(8, 4)
        )
        plt.title("Churn Distribution by Contract (Synthetic Data)")
        plt.xlabel("Contract type")
        plt.ylabel("Share within contract (%)")
        plt.xticks(rotation=15)
        plt.tight_layout()
        plt.savefig(OUTPUT_DIR / "churn_by_contract.png", dpi=160)
        plt.close()

        print(f"\nOutputs saved in: {OUTPUT_DIR.resolve()}")
        print("Reminder: replace synthetic data with an approved dataset before making real-world claims.")


if __name__ == "__main__":
    main()
```